Graph Theory and Network Science

BMED365: Topic List N01–N10

BMED365

Spring 2027

Overview

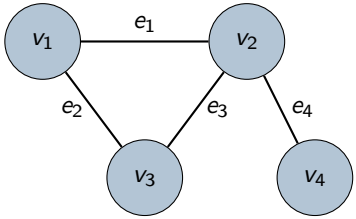
1. **Fundamental Graph Theory**
   - N01: What is a graph? (Nodes and edges)
   - N02: Directed vs. undirected graph
   - N03: Weighted vs. unweighted graph
   - N04: Adjacency matrix
2. **Centrality Measures**
   - N05: Degree Centrality
   - N06: Betweenness Centrality
   - N07: Eigenvector Centrality
   - N08: Clustering Coefficient
3. **Community Detection and Tools**
   - N09: Community Detection and the Louvain algorithm
   - N10: The NetworkX library for Python

N01: What is a graph? (Nodes and edges)

Definition

A **graph** $G = (V, E)$ consists of:

- V: set of **nodes** (vertices)
- E: set of **edges**



Medical example:

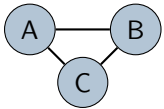
- Nodes = Patients
- Edges = Similarity between patients

$V = \{v_1, v_2, v_3, v_4\}$
$E = \{e_1, e_2, e_3, e_4\}$

N02: Directed vs. undirected graph

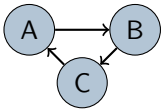
↔ Undirected graph:

- Edges have no direction
- $A - B$ means mutual connection
- Example: Patient Similarity Networks (PSN)



→ Directed graph (digraph):

- Edges have direction
- $A \to B \neq B \to A$
- Example: Gene regulatory networks



Directed Acyclic Graph (DAG)

A directed graph **without cycles** (no path $A \to \cdots \to A$). Used in causal inference, Bayesian networks, and workflow systems.

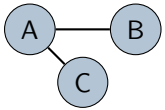
Medical context

PSN (patient similarity networks) are typically **undirected**: similarity between patient A and B is symmetric.

N03: Weighted vs. unweighted graph

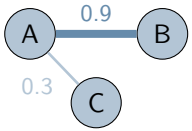
Unweighted graph:

- All edges are "equal"
- Binary: connection or not



Weighted graph:

- Edges have numerical weights
- Represents strength/distance/similarity



In PSN (patient similarity networks)

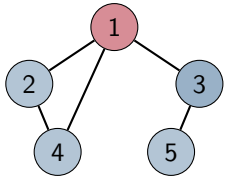
The weight represents **similarity** between patients, calculated from clinical variables.

Higher weight (w) = more similar patients (similar symptoms, biomarkers, disease course, BMI, same gender, ...).

Lower weight = less similar patients.

Often a **threshold value** is used to filter out weak connections (e.g. $w < 0.5$).

N04: Adjacency Matrix



deg: 1→3, 2→2, 3→2, 4→2, 5→1

Adjacency Matrix A:

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

- $A_{ij} = 1$ if edge between node i and j
- $A_{ij} = 0$ otherwise
- Symmetric for undirected graphs

Weighted graph

In weighted graph: $A_{ij} = w_{ij}$ (the weight of the edge)

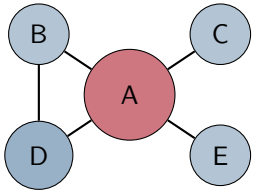
N05: Degree Centrality

Definition

Degree = number of edges to a node

$$C_D(v) = \frac{\deg(v)}{n-1}$$

(normalized: divided by max possible neighbors)



Node	deg	C_D (norm.)
A	4	1.00
B	2	0.50
C	1	0.25
D	2	0.50
E	1	0.25

Interpretation:

- High degree = many connections
- "Hub" in the network
- Easy to calculate

Medical relevance

In PSN: Patients with high degree centrality are similar to many others → "typical" patients.

N06: Betweenness Centrality

Definition

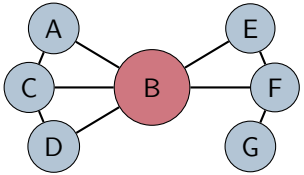
Fraction of shortest paths that pass through a node

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

σ_{st} = number of shortest paths from s to t

Interpretation:

- High betweenness = "bridge" between groups
- Controls information flow
- Critical for network structure



Node	C_B	Node	C_B
A	0.0	E	0.0
B	**15.0**	F	2.0
C	1.0	G	0.0
D	0.0		

Node B is bridge between two clusters

N07: Eigenvector Centrality

Idea

A node is important if it is connected to **other important nodes**

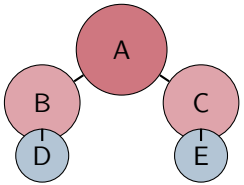
Calculation:

$$x_i = \frac{1}{\lambda} \sum_{j \in N(i)} x_j$$

Matrix form: $Ax = \lambda x$ (eigenvector equation)

Difference from degree:

- Degree: only counts number of neighbors
- Eigenvector: weights neighbors by their importance



A has highest eigenvector centrality

Famous application

Google PageRank is a variant of eigenvector centrality!

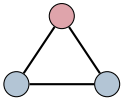
N08: Clustering Coefficient

Definition

How much the neighbors of a node are connected to each other

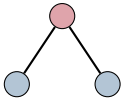
$$C_i = \frac{\text{number of edges between neighbors}}{\text{max possible edges between neighbors}}$$

High clustering ($C = 1$):



"The neighbors know each other"

Low clustering ($C = 0$):



"The neighbors don't know each other"

Medical relevance

High clustering in PSN can indicate tight patient subgroups (phenotypes).

N09: Community Detection and the Louvain Algorithm

Community Detection

Identify **groups** (communities) of densely connected nodes

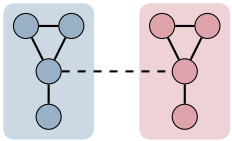
The Louvain algorithm:

1. Start: each node = own group
2. Move nodes to neighbor groups that increase **modularity**
3. Repeat until no improvement
4. Aggregate and repeat hierarchically

Modularity Q**:**

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

i, j=nodes, A_{ij}=adjacency matrix, m=edges, k_i=degree of node i, c_i=community of node i, δ=1 if $c_i = c_j$. $Q > 0.3$: good structure.



Two identified communities

N10: The NetworkX library for Python

NetworkX

Python library to create, manipulate, and analyze graphs

Common operations:

```
import networkx as nx

# Create graph
G = nx.Graph()
G.add_edge('A', 'B', weight=0.8)
G.add_edge('B', 'C', weight=0.5)

# Centrality measures
nx.degree_centrality(G)
nx.betweenness_centrality(G)
nx.eigenvector_centrality(G)
nx.clustering(G)

# Community detection
from community import louvain
partition = louvain.best_partition(G)

# Visualization
nx.draw(G, with_labels=True)

# From adjacency matrix (numpy)
G = nx.from_numpy_array(A)

# Graph properties
G.number_of_nodes()
G.number_of_edges()
nx.density(G)
```

Summary: N01–N10

Graph theory fundamentals:

- Graph $G = (V, E)$: nodes (V) + edges (E)
- Directed vs. undirected
- Weighted vs. unweighted
- Adjacency matrix representation

Centrality measures:

- Degree: number of connections
- Betweenness: bridge role
- Eigenvector: important neighbors

Structural analysis:

- Clustering coefficient: local density
- Community detection: finding groups
- Louvain algorithm: efficient method

Tools:

- NetworkX (Python)
- Integration with NumPy/Pandas

Next steps

Use these concepts to build **patient similarity networks** (PSN)!